

UQABench: Evaluating User Embedding for Prompting LLMs in Personalized Question Answering

Langming Liu[†], Shilei Liu[†], Yujin Yuan, Yizhen Zhang, Bencheng Yan, Zhiyuan Zeng, Zihao Wang, Jiaqi Liu, Di Wang, Wenbo Su, Wang Pengjie, Jian Xu, Bo Zheng
Taobao & Tmall Group of Alibaba

Abstract

Large language models (LLMs) achieve remarkable success in natural language processing (NLP). In practical scenarios like recommendations, as users increasingly seek personalized experiences, it becomes crucial to incorporate user interaction history into the context of LLMs to enhance personalization. However, from a practical utility perspective, user interactions' extensive length and noise present challenges when used directly as text prompts. A promising solution is to compress and distill interactions into compact embeddings, serving as soft prompts to assist LLMs in generating personalized responses. Although this approach brings efficiency, a critical concern emerges: Can user embeddings adequately capture valuable information and prompt LLMs? To address this concern, we propose UQABench, a benchmark designed to evaluate the effectiveness of user embeddings in prompting LLMs for personalization. We establish a fair and standardized evaluation process, encompassing pre-training, fine-tuning, and evaluation stages. To thoroughly evaluate user embeddings, we design three dimensions of tasks: sequence understanding, action prediction, and interest perception. These evaluation tasks cover the industry's demands in traditional recommendation tasks, such as improving prediction accuracy, and its aspirations for LLM-based methods, such as accurately understanding user interests and enhancing the user experience. We conduct extensive experiments on various state-of-the-art methods for modeling user embeddings. Additionally, we reveal the scaling laws of leveraging user embeddings to prompt LLMs. The benchmark is available online at <https://github.com/OpenStellarTeam/UQABench>.

CCS Concepts

• Computing methodologies → Language resources.

Keywords

Large Language Models, Recommendation, Personalization

ACM Reference Format:

Langming Liu[†], Shilei Liu[†], Yujin Yuan, Yizhen Zhang, Bencheng Yan, Zhiyuan Zeng, Zihao Wang, Jiaqi Liu, Di Wang, Wenbo Su, Wang Pengjie, Jian Xu, Bo Zheng. 2018. UQABench: Evaluating User Embedding for Prompting LLMs in Personalized Question Answering. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
<https://doi.org/XXXXXXX.XXXXXXX>

(Conference acronym 'XX). ACM, New York, NY, USA, 10 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

The advent of large language models (LLMs) has revolutionized the field of natural language processing (NLP), showcasing remarkable performance across various tasks, such as machine translation, sentiment analysis, semantic understanding, and multi-turn dialogues [17, 27, 36, 44]. The powerful capabilities of LLMs have the potential to extend to numerous personalized scenarios, including platform search, recommendation, and online advertising, which are highly related to user experience [1, 6, 24]. Specifically, the information cocoons caused by collaborative filtering (CF)-based methods are always a critical challenge that jeopardizes user experience in personalization tasks [25, 28]. With rich worldwide knowledge and diversity, LLMs provide a chance to break the information cocoons of users and bring sufficient personalization [42, 43, 47, 49]. One promising direction LLMs can take to provide personalization is incorporating user history interactions as contextual information [5, 12, 19, 22, 29]. Specifically, the user interaction data are collected from the behavioral interactions (e.g., click, favorite, add to cart, and purchase) between users and items in various scenarios, e.g., searching and recommendation. Therefore, user interaction data can provide abundant information regarding user interests and preferences, a valuable source for providing user-side context for prompting LLMs in personalization.

Despite the promise of enhanced personalization by integrating user interaction history into the context of LLMs, this approach faces significant challenges. User interaction data can be lengthy and noisy [30, 35, 39], making direct incorporation as text prompts impractical in industrial settings. First, the massive number of tokens in interaction sequences (e.g., tens of thousands) slows down the inference speed of LLMs, leading to unacceptable response times and negatively impacting user experience. Technically, the computational complexity of the attention mechanism—the core of LLMs—is enormous, scaling quadratically with the number of tokens N [37]. Moreover, N may even exceed the context limit of LLMs. Second, users' redundant, repeated, and unintentional interactions across various scenarios can mislead LLMs into understanding user interests. Inspired by advancements in recommendations, such as GRU4Rec [8] and SASRec [11], a promising alternative to address this issue is compressing these interactions into more compact user embeddings using encoder models [14, 15, 26]. These user embeddings act as soft prompts, providing “efficient” (i.e., fewer tokens, higher value) contextual information. By extracting high-value information and filtering out irrelevant interactions, this embedding-based approach simultaneously addresses the aforementioned challenges.

While employing user embeddings to prompt LLMs offers a promising and efficient approach to personalization, several significant concerns emerge regarding their effectiveness:

- Can user embeddings capture the necessary information from user interactions and effectively convey it to LLMs?
- When contextualized through user embeddings, can LLMs perform traditional recommendation tasks successfully?
- Do user embeddings serve as strong prompts that guide LLMs to produce personalized responses aligned with user interests?

Unfortunately, previous evaluations primarily concentrate on the traditional recommendation performance [3, 8, 11], such as user-item similarity scores and recall/rank metrics. This focus leaves a gap in comprehensively assessing how user embeddings address these concerns in the LLM era. To bridge this gap, we introduce **UQABench**, a novel Chinese **Benchmark** designed to evaluate the quality of User embeddings in prompting LLMs for personalized Question Answering. The interactive, personalized Q&A paradigm aligns with the LLMs' parameterized nature. Notably, our approach stands out for its standardization and comprehensiveness in evaluating user embeddings.

The standardized evaluation flow of UQABench includes three steps: (1) **Pre-training**: We utilize sufficient user interaction data to pre-train encoder models. (2) **Fine-tuning**: We align learned user embeddings with semantic space through fine-tuning, a step crucial for making LLMs capable of utilizing user embeddings for personalization. (3) **Evaluating**: We evaluate user embeddings in prompting LLMs for task-specific personalized Q&A. To provide a comprehensive assessment, we design three critical tasks that extend beyond conventional evaluations and address the previously mentioned concerns: (1) **Sequence understanding** measures the qualities of user embeddings in prompting LLMs to understand direct features and match features, crucial for determining whether user embeddings can substitute massive feature engineering in the industry. (2) **Action prediction** involves next-item and next-attribute predictions, reframing traditional recommendation tasks within a natural language paradigm. (3) **Interest perception** focuses on modeling user interest, covering long- and short-term interests and interest trajectory, aligning closely with new industrial demands such as enhancing user experience. We evaluate various state-of-the-art user encoder models using UQABench, providing valuable insights for employing user embeddings in the LLM era. Furthermore, we explore the scalability of Transformer-based encoders in prompting LLMs for personalization. Our work seeks not only to validate effective methodologies but also to lay the groundwork for future innovations in user personalization.

The main contributions of our work are three-fold:

- We introduce UQABench, a novel benchmark that comprehensively evaluates the user embeddings in prompting LLMs for personalization. This benchmark uniquely leverages personalized interactive question answering, distinguishing itself from traditional benchmarks.
- We construct a standardized assessing process to ensure evaluation reliability. Additionally, our proposed assessment tasks comprehensively address the effectiveness of user embeddings, encompassing industrial recommender system focuses from both old and new eras.

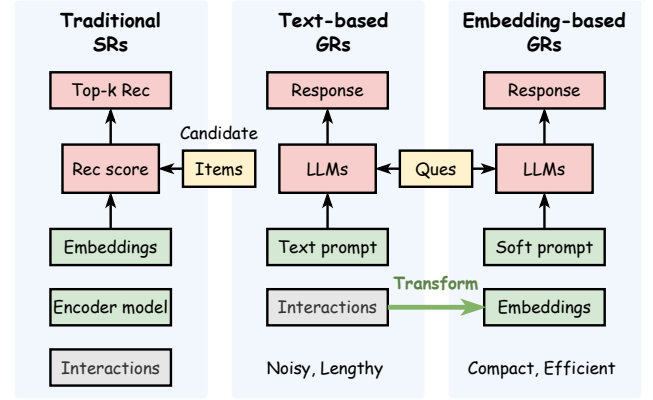


Figure 1: A brief comparison of SRs and GRs.

- The data and code are open source, facilitating further research. Extensive experiments offer novel insights into learning user representations, providing guidelines for optimizing personalization and enhancing user experience in the LLM era.

2 Preliminary

We briefly introduce how both previous and current recommender systems utilize user interactions, focusing on two prevailing branches: sequential recommendations (SRs) and generative recommendations (GRs). The brief comparison is illustrated in Figure 1.

2.1 Sequential Recommendations (SRs)

Given a set of users $\mathcal{U} = \{u_1, u_2, \dots, u_{|\mathcal{U}|}\}$ and a set of items $\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}$, consider that the interaction sequence of user u_i is denoted as $s_i = [v_1^i, v_2^i, \dots, v_{n_i}^i]$. For user u_i , SRs take s_i as input and calculate recommendation scores (e.g., cosine similarity) for candidate items, then output the top- k items most likely to be interacted with in the subsequent time step.

2.2 Generative Recommendations (GRs)

GRs represent a shift towards utilizing LLMs to generate more personalized recommendation results, divided into two main branches:

2.2.1 Text-based GRs. Text-based GRs directly leverage the textual information of interactions. Interacted items' IDs and textual attributes, such as titles, categories, and descriptions, are chronologically placed in pre-designed prompts to serve as user context. Given questions like "What item will the user click next," the LLMs generate responses such as "item j ."

2.2.2 Embedding-based GRs. In contrast, embedding-based GRs transform lengthy, noisy interaction sequences into information-dense user embeddings. These user embeddings, aligned to the semantic space by an adapter, act as soft prompts for LLMs, personalizing them further. Compared to text-based GRs, this approach requires fewer tokens to convey user context, enhancing efficiency and scalability. However, a critical question remains: "Can this approach provide comparable personalized performance to text-based GRs?" This question forms one of the core research focuses of this paper.

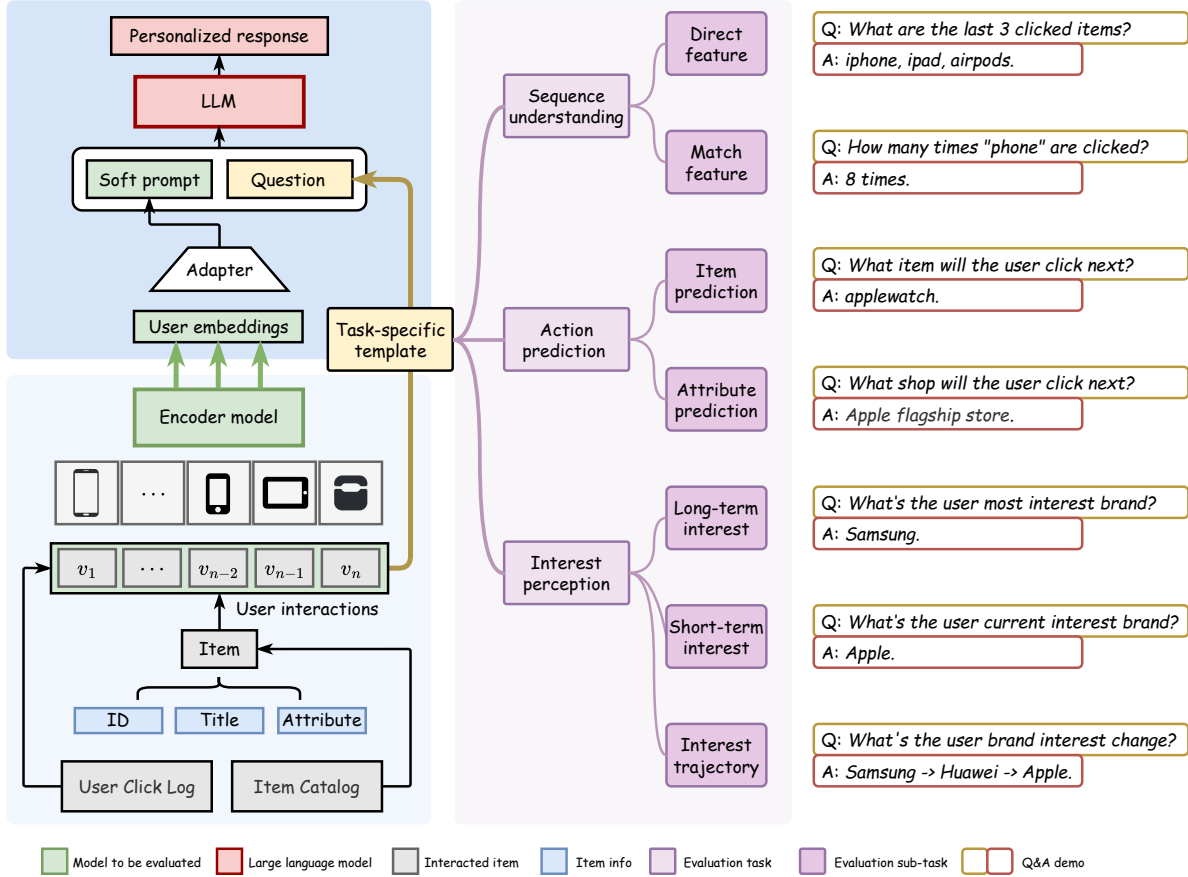


Figure 2: An overview of the standardized evaluation process of user embeddings on UQABench. The left-bottom part demonstrates the encoder model encodes the historical interactions into user embeddings, which requires pre-training. The left-top part shows that user embeddings act as the soft prompt for LLMs to generate personalized responses, which needs fine-tuning. The right side demonstrates the proposed tasks in the evaluating stage and the corresponding Q&A demo.

3 UQABench

The overview of UQABench is shown in Figure 2. Based on the industrial user click log and item catalog, we construct the user interaction data as the evaluation source. The evaluation process comprises three critical steps: **pre-training**, **fine-tuning**, and **evaluating**. First, we pre-train the encoder models on user interactions to learn initial user embeddings. Next, we align these user embeddings to the semantic space through fine-tuning, enabling them to act as soft prompts for LLMs. Finally, we assess the quality of the embeddings by evaluating the performance of LLMs prompted by these embeddings in personalized question answering tasks.

3.1 Source Data

To build a benchmark specialized in user personalization, we require an open source dataset that includes sufficient user interactions and item information. Unfortunately, two crucial reasons hinder us from directly leveraging existing datasets, such as MovieLens, Yelp, and Amazon Review Data. First, such datasets mainly focus on specific fields (e.g., movies, merchants, books, and sports) and

specific tasks (e.g., next-item and rating predictions), jeopardizing evaluation comprehensiveness. Second, these datasets suffer from issues related to information richness, such as limited users and items, insufficient side information, and short interaction sequences, all of which reduce the reliability of the evaluation. To address the issues above, we construct and release a brand-new user interaction dataset as the source data for UQABench. This dataset is collected and processed from **Taobao's User Click Log**. Statistically, it includes 184,520 users, 994,447 items, and 31,317,087 interactions, along with extensive item-side information (e.g., item title, category name, brand name, shop name) sourced from **Taobao's Item Catalog**. To ensure user privacy and comply with regulations, we perform necessary operations, detailed in Section 3.5.1, on data without destroying the sequence patterns. Then, we launch the evaluation flow leveraging the well-prepared source data.

3.2 Stage 1: Pre-training

The first step of evaluation on UQABench is the pre-training stage, which is essential for encoder models to learn to encode noisy and

lengthy user interaction sequences into denser representations (i.e., user embeddings). In this stage, we focus on three critical factors that greatly affect the upper bound of performance: model inputs, training tasks, and training methods. To determine the optimal settings of these three critical parts, we conduct a corresponding ablation study in Section 4.3.1.

3.2.1 Model Inputs. Model inputs are fundamental in pre-training encoder models. The item ID, serving as the unique identifier for each item (e.g., book, movie, music), remains the most critical feature in traditional recommendations. Additionally, side information such as title, category, brand, and shop provides crucial contextual information that enhances recommendation performance. The item side information includes ID-based information (e.g., category ID) and text-based information (e.g., category name). We utilize item ID as the necessary input and consider the ID-based and text-based side information as the optional inputs.

3.2.2 Training Tasks. Determining the appropriate training task is vital for the pre-training stage. Although different encoder models may use various tasks (e.g., sequential or bidirectional order, next prediction or cloze tasks, item or attribute prediction) in the original works, we standardize the training task to ensure fairness and improve efficiency. We adopt the next-item prediction (NIP) [8, 11] as the uniform pre-training task for all encoder models. NIP is a widely used paradigm in sequential recommendations and has proven its effectiveness in learning user representations [3].

3.2.3 Training Methods. Regarding encoder training methods, two common approaches in recommender systems are employed: **supervised learning** and **contrastive learning**. Supervised learning (SL), typically optimized via classification losses such as cross-entropy (CE), achieves strong performance when labeled data is abundant and reliable. However, in real-world scenarios where explicit labels are scarce, and the item pool is vast (leading to data sparsity), SL struggles to generalize. In contrast, contrastive learning (CL) methods such as InfoNCE are preferred in industrial settings. Contrastive learning operates under a self-supervised paradigm: it learns user and item representations by contrasting positive pairs (e.g., user-item interactions) against negative pairs (e.g., randomly sampled non-interactions). This method achieves efficiency via in-batch negative sampling and robustness in cold-start/long-tail scenarios by leveraging intrinsic data structures.

3.3 Stage 2: Fine-tuning

The pre-trained encoders are capable of producing user embeddings, capturing user interests and sequential patterns. Unlike serving collaborative filtering functions (e.g., calculating user-item similarity) in traditional recommendations, user embeddings will act as a soft prompt to activate the personalization of LLMs. However, the discrepancy between the dimensions of the user embeddings and semantic space obstructs direct use. To this end, the adapter is needed to transform user embeddings into word embeddings' dimensions. Additionally, compression methods are adopted to reduce the length of user embeddings, where we consider two approaches: **Mean Pooling** and **Q-Former**, specified in Section 4.3.2. In this way, the user embeddings are converted to several "soft-prompt tokens". A fine-tuning stage of the framework is necessary for (1)

transferring the information encoded in user embeddings to LLMs and (2) generating responses consistent with the answer paradigm to improve evaluation efficiency. We freeze the pre-trained encoders to avoid the knowledge of LLMs overwhelming them and causing catastrophic forgetting of learned user sequential information. We explore two fine-tuning strategies: full fine-tuning (LLM+adapter) and partial fine-tuning (only adapter) in Section 4.3.2.

3.4 Stage 3: Evaluating

Next, we prompt the LLMs with aligned user embeddings to generate personalized responses and evaluate their quality. The critical challenge is scaling up the evaluation while guaranteeing objectivity simultaneously. To this end, we manually design some question templates as seeds according to the interaction data patterns and expertise. This way, massive question prompts (with their ground truths) can be automatically generated based on user historical interactions. We propose three critical evaluation tasks to assess user embeddings from multi-dimensional perspectives: sequence understanding, action prediction, and interest perception, as illustrated in Figure 2. As specified in Table 1, we subdivide each task into two or three sub-tasks for a more fine-grained evaluation and design various task-specific templates for each sub-task. Some details of template designing are mentioned in Section 3.5.2. We then introduce each evaluation task from a conceptual perspective.

3.4.1 Sequence Understanding. Sequence understanding involves using LLMs to extract and restore historical user information from user embeddings. The primary goal of this task is to assess how well user embeddings can serve as a bridge to convey necessary information from user interaction sequences to LLMs. This task is also of significant interest to the industry since it is related to whether user embeddings in the LLM era can substitute for massive feature engineering on the user side. Specifically, for a given user interaction sequence, we can query LLMs for **direct features** like item IDs, titles, and other side information such as category, shop, and brand. For example, we can construct the question prompt such that, "*what are the last three items the user has clicked?*" or, "*what is the brand of the last item the user has clicked?*" In user modeling of industrial recommendation scenarios, the **match features** play a critical role in feature engineering that captures high-order user sequence patterns. To this end, we design questions focusing on match features, defined as the statistical attributes of the user sequence that match the target feature, like item, category, or brand. For example, "*given a category, tell me the times the user has clicked on this category in history.*" Although the questions about match features are more complicated than direct features, it is a practical approach for exploring the upper bound of user embeddings in the sequence understanding task.

3.4.2 Action Prediction. We are interested in how user embeddings can assist LLMs in completing traditional industrial tasks like top- k recommendations and click-through rate (CTR) prediction, which are highly related to revenue. An essential task proposed by us for evaluating these industrial capabilities is action prediction, which includes next-item and next-attribute predictions, the pivotal focus of user sequence modeling. Traditional next-item prediction involves inputting a user interaction sequence into the

Table 1: Details and interpretations of evaluation tasks.

Task	Sub-task	Interpretation
Sequence understanding	Direct feature (DF)	Directly recovering the features in user interaction sequence.
	Match feature (MF)	Counting the statistics of features matching the target feature.
Action prediction	Item prediction (IP)	Predicting the next item the user will most likely interact with.
	Attribute prediction (AP)	Predicting the attribute of the next item the user will most likely interact with.
Interest perception	Long-term interest (LI)	Perceiving the user’s long-term interest category/brand/store.
	Short-term interest (SI)	Perceiving the user’s current interest category/brand/store.
	Interest trajectory (IT)	Detecting the interest changes of the user.

model and outputting the item the user will most likely interact with next time from the candidate items. Two key differences exist between our proposed task and the traditional next-item prediction task. First, in our approach, we use the compressed user sequence, represented by **user embeddings**, as a soft prompt for LLMs to generate predictions rather than directly using the original user sequence. Additionally, we expand the prediction target beyond the item itself to include other attributes (i.e., side information) of the next items, such as title, category, brand, and shop. For example, we may ask, “*what item brand is the user most likely to click at next time?*” Such capability of providing more diverse responses is a significant advantage of LLMs.

3.4.3 Interest Perception. One revolutionary improvement of LLM-based recommendations compared to traditional recommendations should be **introducing remarkable diversity**. Restricted by the training paradigm and CF framework, traditional recommendations will concentrate on popular items and users with frequent interactions. We hope that user embeddings can assist LLM-based approaches in recalling diverse user interest items to improve personalization and enhance user experience. To evaluate the effect of user embeddings in enhancing personalization, we design some unique question prompts to assess how well LLMs prompted by user embeddings know the user’s interest. As the interaction sequence may be noisy and overlong, the user embedding should highlight the core and indispensable parts (e.g., long- and short-term interests) and reveal the user’s interest trajectory. Regarding user **long- and short-term interests**, we build the prompts such as “*the user wants to purchase products from a certain category/brand/store, based on user’s history information, recommends three options,*” and “*what is the user’s recent interest category/brand/store.*” The ground truth of the former is related to the whole interaction sequence, while the latter consists of the recently interacted items. Considering the user’s **interest trajectory**, we may ask, “*has the user’s interest recently changed, and what is the trajectory of interest change.*”

3.5 Technical Details

Besides introducing the main evaluation process, several technical details merit attention, which we discuss below.

3.5.1 Data Preprocessing. First, for privacy protection and compliance regulations, we anonymize the user IDs to prevent locating users and filter out sensitive items. Furthermore, the original user interaction data is lengthy and noisy, and the number of items

is massive, which is inconducive to encoder models’ training efficiency and stability. To address these issues, we conduct necessary data preprocessing on both the user and item sides. On the user side, we filter out users with interaction sequences that are either too long or too short, thereby enhancing the efficiency and stability of mini-batch training. Regarding items, we remove those with fewer than three user interactions to reduce the size of the item pool, significantly improving the training effectiveness of user encoders.

3.5.2 Prompt Designing. We design templates of question prompts to enhance evaluation efficiency and objectivity. Given a task-specific template and user data, we can automatically generate the corresponding question and answer based entirely on user interactions. For instance, a template designed for the direct feature task might be “*what are the last k items the user has clicked?*”, and inputting the interaction sequence $s_i = [v_1, \dots, v_{n_i}]$ of user u_i would automatically yield the answer $v_{n_i-k+1}, \dots, v_{n_i}$. Since it is unrealistic for LLMs to generate complete answers to highly specialized questions, we provide candidates for such questions. Furthermore, we adopt some filter rules to avoid trivial or overly burdensome questions, for example, discarding questions with an answer of 0 (i.e., no match features) in match feature tasks and ensuring k is less than 4 in direct feature tasks. Some necessary instructions are also provided to guide LLMs’ generation, like arranging multiple answers in chronological order for direct feature tasks or sorting the answers by user interest level for interest perception tasks. However, these instructions only play an auxiliary role, as the fine-tuning stage plays the leading role.

3.6 In-depth Analysis: Scaling Law

We aim to reveal the scaling law of using embeddings to prompt LLMs for personalization, which benefits resource distribution in industrial scenarios. Following valuable experiences from LLMs and recommender systems, we focus on two factors: **model size** (of user encoders) and **sequence length** (of interaction data in the pre-training stage). Scaling up the model size is validated to improve model performance in various domains, e.g., NLP [13]. In addition, the previous experience in SRs [3] is that scaling up the sequence length can consistently improve the recalling and ranking performance. We will explore whether such conclusions still hold for the effect of user embeddings in prompting LLMs for personalization. As the core of LLMs, Transformers are proven to possess excellent scaling capabilities. Therefore, our scaling experiments will apply Transformers as the backbone models.

Table 2: Results of evaluating LLMs on UQABench dataset. The overall average (Avg.) score is bold in each row.

Methods	Models	Sequence understanding			Action prediction			Interest perception				Overall Avg.
		DF	MF	Avg.	IP	AP	Avg.	LI	SI	IT	Avg.	
Text-based	Text20	38.25	28.91	37.52	44.81	40.58	42.45	63.82	79.96	62.79	71.10	53.02
	Text50	44.22	32.03	43.27	45.58	38.04	41.39	79.27	<u>89.80</u>	<u>79.63</u>	<u>84.29</u>	59.32
Emb-based	GRU4Rec	33.67	23.44	32.86	34.94	30.02	32.20	46.98	41.33	45.56	43.97	37.15
	SASRec	40.41	24.22	39.14	42.49	35.27	38.47	59.20	53.32	56.39	55.72	45.66
	Mamba4Rec	30.54	18.75	29.62	34.25	28.10	30.83	38.82	34.69	35.64	36.06	32.69
	HSTU	43.29	25.00	41.86	47.30	36.53	41.31	70.85	60.06	65.67	64.45	50.88
	Trm++	47.28	27.34	45.72	48.67	37.35	42.38	75.25	<u>64.14</u>	<u>70.10</u>	<u>68.71</u>	53.88

4 Experiments

We assess the performance of various user embeddings in prompting LLMs for personalized question answering. Specifically, we want to reveal the answers to the following research questions:

- **RQ1:** What is the overall performance of user embeddings in prompting LLMs for personalized question answering?
- **RQ2:** How is the contribution of each component?
- **RQ3:** What is the scaling law of user embeddings in prompting LLMs for personalized question answering?
- **RQ4:** How is the efficiency of embedding-based GRs?

4.1 Experimental Setup

4.1.1 Datasets. We randomly split 31,317,087 interactions from source data into 9 : 1 for the training and test sets. The training set serves in the pre-training and fine-tuning stages. Based on the test set, we generate 7,192 personalized Q&A for the evaluating stage.

4.1.2 Models. The user embeddings to be evaluated are produced by encoder models. We experiment on several state-of-the-art models to comprehensively evaluate embedding-based GRs.

- **GRU4Rec** [8]: leverages the gated recurrent units (GRUs), the famous class in recurrent neural networks (RNNs) to learn the user sequential representation.
- **SASRec** [11]: applies self-attention, the core of transformers, to embed user interests from interaction sequences.
- **Mamba4Rec** [18]: captures user temporal dynamics using state space models (SSMs). Employing a selective mechanism, it identifies important states while ignoring irrelevant ones.
- **HSTU** [46]: redefines recommendations as sequential transduction tasks within GRs. It introduces a novel module, HSTU, specifically designed for industrial recommendation scenarios.
- **Trm++** [46]: is a prominent Transformer variant [36] leverages upgraded architecture accompanied by RoPE [31], SwiGLU.

As illustrated in Figure 2, the user embeddings will serve as the soft prompts to contextualize LLMs to generate personalized responses conforming to user interests. To guarantee fairness, we utilize a uniform backbone LLM, Qwen2.5-3b-instruct [44], for all encoder models. We also consider text-based GRs for comparison as follows.

- **Text20 & Text50:** leverages user sequence of 20 and 50 interactions, respectively, as text prompt of LLMs to generate responses.

4.1.3 Implementation Details. Following the previous research [11, 32], we initialize the trainable parameters using Gaussian distribution. We employ the AdamW optimizer [21] with a learning rate of 1e-4 and mini-batch size 256 to optimize all models. Based on the configurations in prior studies [11, 32, 48] and maintaining comparability, we ensure that all model sizes are on the same order of magnitude (approximately 20M) and globally adjust the hyperparameters. For instance, regarding GRU4Rec, we utilize a 3-layer GRU module with a hidden size of 1024, and for SASRec, the number of layers is set to $L = 6$, each with $h = 8$ attention heads and dimension size $d = 1024$, with an inner size (i.e., of FNN layers) of 1536. We utilize $N = 512$ as the global sequence length. We zero-pad sequences shorter than N and truncate those longer than N . The input and output layers share an item embedding matrix, and the dropout rate is set to 0.1 to alleviate overfitting. The maximum numbers of epochs for pre-training and fine-tuning are 10 and 3, respectively. Our implementations¹ are conducted in the environment using PyTorch 2.4.0 and Python 3.10.13 with 8× GPUs.

4.2 Overall Comparison (RQ1)

We evaluate several advanced user encoder models on proposed UQABench following the standardized evaluation process (Figure 2) and report the overall performances in Table 2. In general, UQABench shows excellent discriminative power for different models, demonstrating its evaluation effectiveness. We draw some interesting observations as follows:

- **Embedding-based GRs vs. Text-based GRs:** The best model of embedding-based GRs (i.e., using Trm++) can achieve satisfactory performance on par with text-based GRs, showing the effectiveness of this method in personalization. Remarkably, embedding-based GRs surpass text-based counterparts in item prediction tasks since they are pre-trained to capture more prior information. However, embedding-based GRs perform significantly worse than text-based methods in interest perception tasks, indicating room for improvement in the framework.
- Among embedding-based methods, HSTU and Trm++ present superior performance, providing high-quality user embeddings to prompt LLMs to give relatively satisfied responses, conforming

¹We release code and data at <https://github.com/OpenStellarTeam/UQABench> to encourage reproducibility and facilitate further research on this topic.

Table 3: Ablation in pre-training stage.

Training	Inputs	SU	AP	IP	Avg.
SL	Full Info	45.72	42.38	68.71	53.88
	w/o Text	42.08	39.27	66.84	51.16
	w/o Side	39.55	37.83	61.47	47.87
CL	Full Info	42.41	39.25	62.37	49.40
	w/o Text	37.06	35.35	58.02	44.99
	w/o Side	36.50	32.84	51.96	41.47

Table 4: Ablation in fine-tuning stage.

Integration	Strategy	SU	AP	IP	Avg.
Mean Pooling	Full FT	43.70	38.83	72.70	53.75
	Adapter	45.72	42.38	68.71	53.88
Q-Former	Full FT	45.35	44.37	69.89	55.00
	Adapter	31.15	29.92	31.80	30.96

to user preferences. The model performances exhibit anisotropy, meaning the performance of models on each task is not linearly correlated. For example, SASRec performs close to HSTU on sequence understanding and action prediction tasks yet far worse than HSTU on interest perception.

- Regarding evaluation tasks, most encoder models **perform significantly well in interest perception tasks**, showcasing that the LLM-based system is inclusive and diverse to perceive user personalized interest instead of only popular items. The models' performance on action prediction tasks is also quite acceptable, suggesting that the embedding-based GRs paradigm has potential in traditional recommendation tasks. Nevertheless, the sequence understanding task results are not good enough, especially the sub-task match feature, reflecting that the current framework is immature to completely substitute the industrial feature engineering and needs future improvements.

4.3 Ablation Study (RQ2)

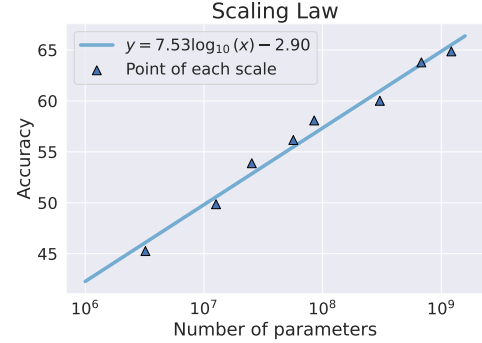
To guarantee evaluation reliability and fairness, we conduct an ablation study in terms of pre-training and fine-tuning stages to determine the optimal settings of the experiments.

4.3.1 Pre-training. The two critical factors in pre-training are model inputs and training methods. We consider three variants of model inputs: (1) **Full Info**: inputs entire information, including item ID and side information (e.g., category ID and category name), (2) **w/o Text**: leverages the item ID and ID-based side information (e.g., category ID), meaning without textual information, and (3) **w/o Side**: only uses item ID without side information. Furthermore, we consider two training methods: (1) **SL**: uses the CE loss to supervise training, and (2) **CL**: utilizes InfoNCE loss to perform contrastive training on encoders. According to the results in Table 3, we observe that the combination of **Full Info** + **SL** yields the best performance. Consequently, we utilize such a setting for our experiments.

4.3.2 Fine-tuning. We explore the optimal settings in fine-tuning. We compare two compression approaches in the adapter: (1) **Mean**

Table 5: The scalability in terms of model size.

Model size	Architecture (l, d, h)	Evaluation tasks			
		SU	AP	IP	Avg.
3.2M	(4, 256, 4)	39.00	34.74	58.16	45.26
13M	(4, 512, 8)	38.62	37.08	67.55	49.86
25M	(8, 512, 8)	45.72	42.38	68.71	53.88
57M	(8, 768, 12)	45.29	42.15	74.74	56.16
85M	(12, 768, 12)	47.12	44.32	76.48	58.08
303M	(24, 1024, 8)	50.43	46.65	77.30	60.01
680M	(24, 1536, 12)	52.20	48.67	83.74	63.78
1.2B	(24, 2048, 16)	53.98	49.16	84.97	64.86

**Figure 3: Scaling law in terms of the model size.**

Pooling: uses a mean pooling layer to compress user sequence embeddings into a single embedding, (2) **Q-Former**: leverages Q-Former to generate fixed-length embeddings, set to 16 by default. Moreover, we consider two fine-tuning strategies: (1) **Full FT**: fine-tunes the adapter and LLM together, and (2) **Adapter**: fine-tunes only the adapter. Table 4 shows that the setting of **Full FT** + **Q-Former** achieves optimal performance. However, we observe that the Q-Former method is sensitive to the hyperparameters (e.g., learning rate), leading to instability (i.e., emerging NaN loss). Therefore, we opt for a more stable configuration, **Mean Pooling** + **Adapter**, as our experiment's essential fine-tuning configuration for consistency and reliability.

4.4 Scalability Study (RQ3)

We explore the scalability of user embeddings in prompting LLMs for personalization. We employ Trm++ as the backbone encoder and follow the proposed evaluation process. Our scalability study considers two critical factors of encoders: model size and sequence length. The experimental details and results are as follows.

4.4.1 Model Size. We scale the model size of the encoders, ranging from 3M to 1.2B. The results are reported in Table 5, where we specify the architectural details, i.e., the number of layers l , dimension size d , and the number of heads h , of the encoder. The experimental results show a clear scaling law between model performance and model size, illustrated in Figure 3. Such a result is significant to the application scenario: We can strengthen encoder models offline (i.e., scaling up model size) to consistently enhance the personalization performance of LLMs in the online environment without impairing the inference efficiency.

Table 6: The scalability in terms of sequence length.

Sequence length	Evaluation tasks			
	SU	AP	IP	Avg.
32	27.78	30.83	38.79	33.38
64	37.21	35.67	53.75	43.40
128	40.88	38.64	63.97	49.49
256	42.41	41.23	69.33	52.96
512	45.72	42.38	68.71	53.88

Table 7: Efficiency comparison on UQABench.

Methods	Models	Performance	Avg. Tokens
Text-based	Text20	53.02	1171.23
	Text50	59.32	2498.19
Emb-based	Trm-M	53.88	133.28
	Trm-Q	55.00	148.28

4.4.2 Sequence Length. We also investigate the impact of sequence length on personalized performance by adjusting this parameter during pre-training. Specifically, we experiment with various maximum sequence lengths ranging from 32 to 512 and present the corresponding outcomes in Table 6. We find that while embedding-based GRs enhance performance as the sequence length increases, this improvement rate diminishes beyond a certain length (e.g., 256). This trend is reasonable, given that the actual length of user interactions is generally limited.

4.5 Efficiency Study (RQ4)

We compare the efficiency of text-based and embedding-based GRs to provide insights into deployment scenarios. When deploying LLMs for online serving, the number of tokens is critical to inference efficiency. Therefore, we report the average number of tokens for the methods tested, as shown in Table 7, where **Trm-M** and **Trm-Q** correspond to Trm++ leveraging two compression methods (**Mean Pooling** and **Q-former**). The results indicate that embedding-based GRs significantly reduce the number of tokens compared to their text-based counterparts, achieving increased efficiency by a factor of 8× to 19×. This efficiency gain is achieved by compressing the user context, i.e., interactions, into fewer tokens (1 for Trm-M and 16 for Trm-Q). In the future, refining question prompts may substantially increase efficiency as they occupy relatively more tokens than soft prompts (i.e., user embeddings).

5 Related Work

5.1 Sequential Recommendations

Sequential recommendations (SRs) learn the user representation from the historical interaction sequence, then calculate the recommendation scores of candidate items and choose top- k as the recommendation result [3, 38]. Previous works try various deep learning modules to enhance user modeling performance. For example, GRU4Rec [8] leverages the GRU layers, Caser [33] uses the CNN layers, and HGN [23] adopts GLU layers. The recent

research direction of SRs gradually converges to prevalent Transformer [37]. The Transformer-based methods, such as SASRec [11], BERT4Rec [32], FDSA [48], BST [2], and CORE [9], leverage attention layers to capture the node correlation within the interaction sequence, which remarkably elevates the representation learning. Despite the performance effectiveness of Transformer-based methods, the main challenges are efficiency [34]. The attention module of Transformers brings quadratic computational complexity with sequence length, which is unrealistic for deployment in industrial recommender systems that need low inference time. The most recent works, such as LinRec [20], LRURec [45], Mamba4Rec [18], provide efficient solutions that can par with or even outperform Transformer-based methods. However, one essential issue is that the CF framework of SRs may lead to information cocoons and jeopardize user experience.

5.2 Generative Recommendations

Generative recommendations (GRs) leverage the generative capabilities of LLMs to generate more personalized and diverse results besides candidate items [42, 43, 47, 49]. The mainstream of GRs, text-based GRs, leverage the text information in user interaction sequences as the user context to prompt LLMs to generate personalized results [5, 12, 19, 22, 29]. Some frequently used prompting methods in the LLM area can be adapted to text-based GRs, such as in-context learning (ICL) [4], and chain-of-thought (CoT) [41]. The ICL-based methods [16, 19] incorporate the task description and in-context demonstrations (i.e., few-shot). While the CoT-based approaches [10, 40] design task-specific reasoning steps to assist LLMs in generating personalized responses step-by-step. Nevertheless, employing the whole user interaction sequence as the text context of LLMs is impractical and unscalable. A regular user interaction sequence with hundreds of items can produce 10k to 100k tokens, which may cause an unacceptably high inference time and cost and even exceed the LLMs' context window limit. Therefore, a line of GRs tries to transform the user interaction sequence into a compact form, user embedding, to softly prompt LLMs for personalization [7, 14, 15, 26]. The embedding-based GRs highly reduce the token number compared to directly using text information, improving the efficiency and scalability of GRs.

6 Conclusion

In the LLM era, the user interaction data provide a rich source to contextualize LLM to be more personalized, which implies a promising recommendation paradigm, GRs. By compressing the interaction sequence into a more compact form, user embeddings, the embedding-based GRs outperform text-based GRs with less inference pressure and are friendly to real-world deployments. To address the concerns about the effectiveness of embedding-based GRs, we propose a novel benchmark, UQABench, evaluating user embeddings from the personalization perspective. We propose a standardized three-stage evaluation process, including pre-training, fine-tuning, and evaluating. Furthermore, to comprehensively assess the quality of user embedding, we design three evaluation tasks and corresponding sub-tasks. We extensively experiment on state-of-the-art encoder models to explore the embedding-based GRs' effectiveness and provide some insights.

References

- [1] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1007–1014.
- [2] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior sequence transformer for e-commerce recommendation in alibaba. In *Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data*. 1–4.
- [3] Hui Fang, Danning Zhang, Yiheng Shu, and Guibing Guo. 2020. Deep learning for sequential recommendation: Algorithms, influential factors, and evaluations. *ACM Transactions on Information Systems (TOIS)* 39, 1 (2020), 1–42.
- [4] Tianyu Gao, Adam Fisch, and Danqi Chen. 2020. Making pre-trained language models better few-shot learners. *arXiv preprint arXiv:2012.15723* (2020).
- [5] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM Conference on Recommender Systems*. 299–315.
- [6] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval augmented language model pre-training. In *International conference on machine learning*. PMLR, 3929–3938.
- [7] Liam Hebert, Krishna Sayana, Ambarish Jash, Alexandros Karatzoglou, Sukhdeep Sodhi, Sumanth Doddapaneni, Yanli Cai, and Dima Kuzmin. 2024. PERSOMA: PERsonalized SOft ProMpt Adapter Architecture for Personalized Language Prompting. *arXiv preprint arXiv:2408.00960* (2024).
- [8] B Hidasi. 2015. Session-based Recommendations with Recurrent Neural Networks. *arXiv preprint arXiv:1511.06939* (2015).
- [9] Yupeng Hou, Binbin Hu, Zhiqiang Zhang, and Wayne Xin Zhao. 2022. Core: simple and effective session-based recommendation within consistent representation space. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. 1796–1801.
- [10] Xu Huang, Jianxun Lian, Yuxuan Lei, Jing Yao, Defu Lian, and Xing Xie. 2023. Recommender ai agent: Integrating large language models for interactive recommendations. *arXiv preprint arXiv:2308.16505* (2023).
- [11] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [12] Wang-Cheng Kang, Jianmo Ni, Nikhil Mehta, Maheswaran Sathiamoorthy, Lichan Hong, Ed Chi, and Derek Zhiyuan Cheng. 2023. Do llms understand user preferences? evaluating llms on user rating prediction. *arXiv preprint arXiv:2305.06474* (2023).
- [13] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [14] Lei Li, Yongfeng Zhang, and Li Chen. 2023. Personalized prompt learning for explainable recommendation. *ACM Transactions on Information Systems* 41, 4 (2023), 1–26.
- [15] Lei Li, Yongfeng Zhang, and Li Chen. 2023. Prompt distillation for efficient llm-based recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 1348–1357.
- [16] Zhiyu Li, Yanfang Chen, Xuan Zhang, and Xun Liang. 2023. Bookgpt: A general framework for book recommendation empowered by large language model. *Electronics* 12, 22 (2023), 4654.
- [17] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).
- [18] Chengkai Liu, Jianghao Lin, Jianling Wang, Hanzhou Liu, and James Caverlee. 2024. Mamba4rec: Towards efficient sequential recommendation with selective state space models. *arXiv preprint arXiv:2403.03900* (2024).
- [19] Junling Liu, Chao Liu, Peilin Zhou, Renjie Lv, Kang Zhou, and Yan Zhang. 2023. Is chatgpt a good recommender? a preliminary study. *arXiv preprint arXiv:2304.10149* (2023).
- [20] Langming Liu, Liu Cai, Chi Zhang, Xiangyu Zhao, Jingtong Gao, Wanyu Wang, Yifu Lv, Wenqi Fan, Yiqi Wang, Ming He, et al. 2023. Linrec: Linear attention mechanism for long-term sequential recommender systems. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 289–299.
- [21] I Loshchilov. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).
- [22] Hanjia Lyu, Song Jiang, Hanqing Zeng, Yinglong Xia, Qifan Wang, Si Zhang, Ren Chen, Christopher Leung, Jiajie Tang, and Jiebo Luo. 2023. Llm-rec: Personalized recommendation via prompting large language models. *arXiv preprint arXiv:2307.15780* (2023).
- [23] Chen Ma, Peng Kang, and Xue Liu. 2019. Hierarchical gating networks for sequential recommendation. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 825–833.
- [24] Aashiq Muhamed, Iman Keivanloo, Sujana Perera, James Mracek, Yi Xu, Qingjun Cui, Santosh Rajagopalan, Belinda Zeng, and Trishul Chilimbi. 2021. CTR-BERT: Cost-effective knowledge distillation for billion-parameter teacher models. In *NeurIPS Efficient Natural Language and Speech Processing Workshop*.
- [25] Tien T Nguyen, Pik-Mai Hui, F Maxwell Harper, Loren Terveen, and Joseph A Konstan. 2014. Exploring the filter bubble: the effect of using recommender systems on content diversity. In *Proceedings of the 23rd international conference on World wide web*. 677–686.
- [26] Lin Ning, Luyang Liu, Jiaxing Wu, Neo Wu, Devora Berlowitz, Sushant Prakash, Bradley Green, Shawn O'Banion, and Jun Xie. 2024. User-LLM: Efficient LLM Contextualization with User Embeddings. *arXiv preprint arXiv:2402.13598* (2024).
- [27] OpenAI. 2023. GPT-4 technical report. (2023).
- [28] Eli Pariser. 2011. *The filter bubble: What the Internet is hiding from you*. Penguin Press.
- [29] Aleksandr V Petrov and Craig Macdonald. 2023. Generative sequential recommendation with gptrec. *arXiv preprint arXiv:2306.11114* (2023).
- [30] Yuqi Qin, Pengfei Wang, and Chenliang Li. 2021. The world is binary: Contrastive learning for denoising next basket recommendation. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 859–868.
- [31] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2024), 127063.
- [32] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.
- [33] Jiaxi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the eleventh ACM international conference on web search and data mining*. 565–573.
- [34] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2022. Efficient transformers: A survey. *Comput. Surveys* 55, 6 (2022), 1–28.
- [35] Changxin Tian, Yuexiang Xie, Yaliang Li, Nan Yang, and Wayne Xin Zhao. 2022. Learning to denoise unreliable interactions for graph collaborative filtering. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. 122–132.
- [36] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [37] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
- [38] Shoujin Wang, Liang Hu, Yan Wang, Longbing Cao, Quan Z Sheng, and Mehmet Orgun. 2019. Sequential recommender systems: challenges, progress and prospects. *arXiv preprint arXiv:2001.04830* (2019).
- [39] Wenjie Wang, Fuli Feng, Xiangnan He, Liqiang Nie, and Tat-Seng Chua. 2021. Denoising implicit feedback for recommendation. In *Proceedings of the 14th ACM international conference on web search and data mining*. 373–381.
- [40] Yancheng Wang, Ziyang Jiang, Zheng Chen, Fan Yang, Yingxue Zhou, Eunah Cho, Xing Fan, Xiaojiang Huang, Yanbin Lu, and Yingzhen Yang. 2023. Recmind: Large language model powered agent for recommendation. *arXiv preprint arXiv:2308.14296* (2023).
- [41] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [42] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2024. A survey on large language models for recommendation. *World Wide Web* 27, 5 (2024), 60.
- [43] Lanling Xu, Junjie Zhang, Bingqian Li, Jinpeng Wang, Mingchen Cai, Wayne Xin Zhao, and Ji-Rong Wen. 2024. Prompting large language models for recommender systems: A comprehensive framework and empirical analysis. *arXiv preprint arXiv:2401.04997* (2024).
- [44] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671* (2024).
- [45] Zhenrui Yue, Yueqi Wang, Zhankui He, Huimin Zeng, Julian McAuley, and Dong Wang. 2024. Linear recurrent units for sequential recommendation. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 930–938.
- [46] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).
- [47] Junjie Zhang, Ruobing Xie, Yupeng Hou, Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Recommendation as instruction following: A large language model empowered recommendation approach. *ACM Transactions on Information Systems* (2023).

- [48] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S Sheng, Jiajie Xu, Deqing Wang, Guanfeng Liu, and Xiaofang Zhou. 2019. Feature-level Deeper Self-Attention Network for Sequential Recommendation.. In *IJCAL*. 4320–4326.
- [49] Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, et al. 2023. Recommender systems in the era of large language models (llms). *arXiv preprint arXiv:2307.02046* (2023).